PG BootCamp Russia 2026 Moscow

PGBootCamp.ru

pg_track_optimizer

Ищем проблемы планирования запросов **ДО** их воздействия на производительность

Андрей Лепихов

PostgreSQL contributor · pgEdge

GitHub

LinkedIn

substack

PLAN ANALYSIS

Key characteristics:

MIT Licence

Online upgrade ready

pg_track_optimizer / FIND THE BAD ESTIMATES

Andrei Lepikhov

PostgreSQL разработчик

pgEdge

lepihov@gmail.com

github.com/danolivo

pgedge.com

О себе

Профессиональный опыт

10 лет в PostgreSQL internals — планировщик запросов, optimizer, executor. Контрибьютор upstream PostgreSQL, фокус на оценке стоимости планов запросов и адаптивной обработке запросов.

Текущая работа · pgEdge

Разработка распределённого, active-active PostgreSQL. Эффективное планирование adhoc запросов и комплексных, шаблонных запросов. Работа над репликацией, разрешением конфликтов и multi-region Postgres.

Open Source и исследования

Основные интересы: адаптивная статистика, адаптивная оптимизация запросов, перепланирование, параллельное выполнение. Активный контрибьютор PostgreSQL community.

Типичная БД и запросы

- Кол-во queryid - 115502
- Длина запроса: 33 - 126к символов
- Execution time: 1e-3 - 26с
- Query Executions: 1 - 5075
- Blocks read: 0 - 3560027 (~28 МБ)
- Plan nodes: 1 - 195
- Количество ‘never executed’ нод в плане: 0 - 52

```
SELECT
T1.Fld100343RRef,
(CAST(CAST((T16.Fld100345_* T1.Fld100346_) AS NUMERIC(28, 8)) / (T16.Fld100346_* T1.Fld100345_) AS NUMERIC(28, 8)))
FROM (SELECT
T15._Fld100344RRef AS Fld100344RRef,
T15._Fld100343RRef AS Fld100343RRef,
T15._Fld100346 AS Fld100346_,
T15._Fld100345 AS Fld100345_
FROM (SELECT
T3._Fld100343RRef AS Fld100343RRef,
T3._Fld100344RRef AS Fld100344RRef,
MAX(T3._Period) AS MAXPERIOD_
FROM _InfoRg100342 T3
WHERE ((T3._Fld2488 = CAST(0 AS NUMERIC))) AND (T3._Period <= '2023-12-04 12:49:02'::timestamp AND (EXISTS(SELECT
1
FROM pg_temp.tt3319 T4
LEFT OUTER JOIN _Reference101 T5
ON (T4._Q_001_F_007RRef = T5._IDRRef) AND (T5._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference395 T6
ON (T4._Q_001_F_005RRef = T6._IDRRef) AND (T6._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference366X1 T7
ON (T4._Q_001_F_006RRef = T7._IDRRef) AND (T7._Fld2488 = CAST(0 AS NUMERIC))
WHERE ((1=1 OR 1=0) AND (T4._Q_001_F_001RRef =
'\\216@\\030f\\332\\261R\\333\\021\\356\\222\\212\\342\\004z\\226':bytea) AND (T5._Fld53293RRef =
'\\214\\215O\\271\\351\\244\\310\\000B<\\227\\003(~\\356e'::bytea) AND ((T7._Fld59861RRef <>
'\\201F\\\\nu\\257Q\\301Fp\\027O\\211\\214\\001('::bytea) OR (T7._Fld59861RRef =
'\\201F\\\\nu\\257Q\\301Fp\\027O\\211\\214\\001('::bytea))) AND ((T3._Fld100343RRef = T5._Fld53297RRef) AND
(T3._Fld100344RRef = T6._Fld101340RRef))
UNION ALL SELECT
1
FROM pg_temp.tt3319 T8
LEFT OUTER JOIN _Reference101 T9
ON (T8._Q_001_F_007RRef = T9._IDRRef) AND (T9._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference101 T10
ON (T9._Fld53299RRef = T10._IDRRef) AND (T10._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference395 T11
ON (T8._Q_001_F_005RRef = T11._IDRRef) AND (T11._Fld2488 = CAST(0 AS NUMERIC))
LEFT OUTER JOIN _Reference366X1 T12
ON (T8._Q_001_F_006RRef = T12._IDRRef) AND (T12._Fld2488 = CAST(0 AS NUMERIC))
WHERE ((1=1 OR 1=0) AND (T8._Q_001_F_001RRef =
'\\216@\\030f\\332\\261R\\333\\021\\356\\222\\212\\342\\004z\\226':bytea) AND (T9._Fld53293RRef =
'\\214\\215O\\271\\351\\244\\310\\000B<\\227\\003(~\\356e'::bytea) AND (T10._Fld53293RRef =
'\\214\\215O\\271\\351\\244\\310\\000B<\\227\\003(~\\356e'::bytea) AND ((T12._Fld59861RRef <>
'\\201F\\\\nu\\257Q\\301Fp\\027O\\211\\214\\001('::bytea) OR (T12._Fld59861RRef =
'\\201F\\\\nu\\257Q\\301Fp\\027O\\211\\214\\001('::bytea))) AND ((T3._Fld100343RRef = T10._Fld53297RRef) AND
(T3._Fld100344RRef = T11._Fld101340RRef));
```

Performance view

Что хотим знать

1. Есть ли потенциал вместить больше TPS на том же сервере?
2. Стоит ли обновлять статистику таблиц?
3. Стоит ли тратить время на оптимизацию запросов и улучшение настроек оптимизатора?
4. Не станет ли хуже, если профиль нагрузки изменится?

Предпосылки

- AQO - Adaptive Query Optimiser - помогал нестабильно
- Replan, SwitchJoin, etc - слишком дорог, если потенциальный полезный эффект незначителен
- pg_stat_statements - отвечает на вопрос “что грузит систему больше всего сейчас”
- Thanks to AWS committers, Postgres теперь позволяет легковесным расширениям использовать shared memory и shared hash tables.
- Появились AI-помощники

Как оценить потенциал системы

1. Величина ошибки планирования (rows estimated / rows actual)
   стоит колдовать со статистикой таблиц?
2. Селективность выборки из БД (Rows Filtered)
   а хватает ли нам индексов?
3. Вклад Subplan’ов в трудоемкость выполнения запросов
   стоит ли переписать запросы?
4. *Спиллинг промежуточных результатов*
   стоит ли выделять запросам больше памяти?
5. Стабильность планов запросов
   может быть generic план был бы хорошей идеей?

Собираем технологии

1. *Metrics* - что и как измерять?
2. Core Hooks - минимальный набор для сбора статистики
3. *Shared Hash Table* - чтобы регистрировать метрики
4. *Shared Memory Segment* - для хранения текстов запросов
5. *File Storage* - пережить рестарт и не схлопнуть производительность
6. *User Interface* - как управлять и анализировать?
7. *Extension module* - сборка, релизы, политика апдейтов

pg_track_optimizer

Выявляет неоптимальные планы запросов через отслеживание оценок кардинальности

PostgreSQL 17+

Режимы

disabled	normal	forced

danolivo/pg_track_optimizer

~1–2% накладных расходов (forced mode)

Архитектура

SQL запрос
передан в PostgreSQL

→

Хук ExecutorStart
Включить instrumentation (rows, timing, page access counters)

→

Хук ExecutorRun
INSTRUMENT_ROWS TIMER · BUFFERS

→

Хук ExecutorEnd
plan_error() обход дерева PlanState

→

DSA хэш-таблица
Общая память статистика запросов

↓ *сброс на диск (опционально)*

Основные файлы

- pg_track_optimizer.c – хуки и DSA реестр
- plan_error.c – обход дерева и агрегация ошибок
- rstats.c – тип онлайн-статистики (Welford)

Метрики ошибок (на запрос)

- avg_error – простое среднее по узлам
- rms_error – среднеквадратичное среднее
- twa_error – взвешенное по времени среднее
- wca_error – взвешенное по стоимости среднее

SQL интерфейс

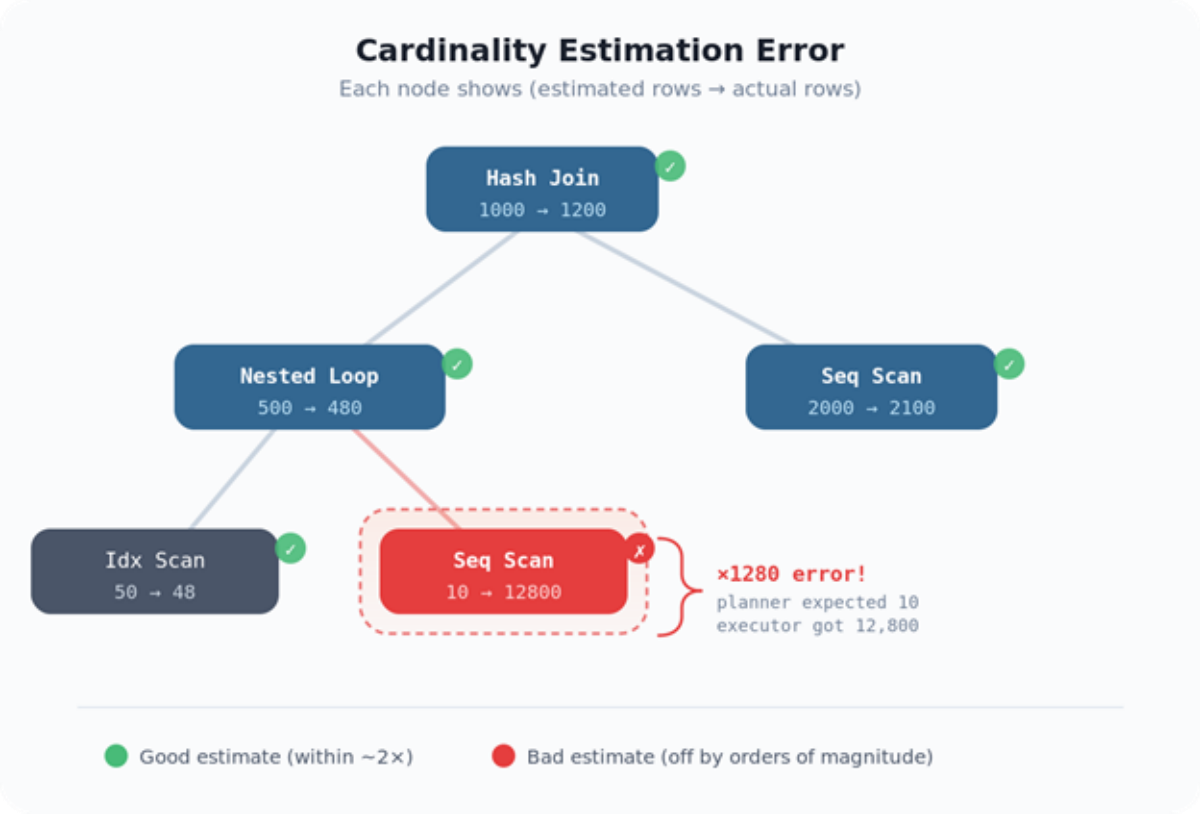
- pg_track_optimizer – представление со статистикой
- pg_track_optimizer() – «сырые» объекты RStats
- pg_track_optimizer_flush() – сброс на диск (CRC32C)

Дополнительные параметры:

- Паттерны блочного I/O
- Спиллинг на диск (work_mem)
- Фактор фильтрации SCAN
- Фактор фильтрации JOIN
- Фактор стоимости SubPlan
- Количество JOINов

Критерий 1: Ошибка оценки кардинальности

Cardinality estimation error



Формула для ошибки кардинальности

$$error = \left(\sum_{nodes} \left| \log \left(\frac{actual\ rows}{planned\ rows} \right) \right| \times normaliser \right) / \ nnodes$$

Виды критерия:

- Простой:
  normaliser = 1
- execution-time based:
  normaliser = node_time / query_total_time
- cost-based:
  node_totalcost / query_totalcost

Как это работает (тест Join Order Benchmark)

Пересечение TOP-10 запросов по критериям average и time-based error:

```
SELECT avg.* FROM (
  SELECT queryid,query,avg_avg,twa_avg
  FROM pg_track_optimizer
  ORDER BY avg_avg DESC LIMIT 10) AS avg
JOIN
(
  SELECT queryid,avg_avg,twa_avg
  FROM pg_track_optimizer
  ORDER BY twa_avg DESC LIMIT 10) AS twa
USING (queryid);

/*
       queryid        |      query      | avg_err | twa_err
----------------------+-----------------+---------+---------
 -6836119621424365027 | /* 28a.sql */  |    3.91 |    2.16
  3631768377448547020 | /* 29c.sql */  |    3.77 |    2.51
 -8544699698054799639 | /* 22d.sql */  |    3.48 |    2.09
  4682104415514032107 | /* 7c.sql */   |    3.48 |    2.18
(4 rows)
*/
```

То же самое для average и cost-based error:

```
/*
       queryid        |      query      | avg_err | wca_err
----------------------+-----------------+---------+---------
 -6836119621424365027 | /* 28a.sql */  |    3.91 |    2.81
  3631768377448547020 | /* 29c.sql */  |    3.77 |    2.71
 -8544699698054799639 | /* 22d.sql */  |    3.48 |    2.49
 -3799596786824522319 | /* 17b.sql */  |    3.48 |    2.23
  4682104415514032107 | /* 7c.sql */   |    3.48 |    2.61
(5 rows)
*/
```

Пересечение трёх критериев:

```
/*
       queryid        |      query      | avg_err | twa_err | wca_err
----------------------+-----------------+---------+---------+---------
 -8544699698054799639 | /* 22d.sql */  |    3.48 |    2.09 |    2.49
 -6836119621424365027 | /* 28a.sql */  |    3.91 |    2.16 |    2.81
  3631768377448547020 | /* 29c.sql */  |    3.77 |    2.51 |    2.71
  4682104415514032107 | /* 7c.sql */   |    3.48 |    2.18 |    2.61
(4 rows)
*/
```

Заглянем в запрос 28a.sql …

1.	0.000	0.000	↑ 1.0	1	→ Aggregate (cost=755,900.71..755,900.72 rows=1 width=96) (actual rows=1.00 loops=1)
2.	0.000	0.000	↓ 16,308.0	1	→ Nested Loop (cost=728,106.12..755,900.70 rows=1 width=48) (actual rows=16,308.00 loops=1)
3.	0.000	0.000	↓ 16,308.0	1	→ Nested Loop (cost=728,105.99..755,900.53 rows=1 width=52) (actual rows=16,308.00 loops=1)
4.	0.000	0.000	↓ 75,240.0	1	→ Gather (cost=728,105.57..755,898.46 rows=1 width=56) (actual rows=75,240.00 loops=1) Workers Planned: 4 Workers Launched: 0
5.	0.000	0.000	↓ 75,240.0	1	→ Parallel Hash Join (cost=728,105.56..755,898.45 rows=1 width=56) (actual rows=75,240.00 loops=1) Hash Cond: (mc.movie_id = t.id)
6.	0.000	0.000	↓ 4.0	1	→ Parallel Seq Scan on movie_companies mc (cost=0.00..25,346.82 rows=652,282 width=8) (actual rows=2,609,129.00 loops=1)
7.	0.000	0.000	↓ 2,868.0	1	→ Parallel Hash (cost=728,105.55..728,105.55 rows=1 width=72) (actual rows=2,868.00 loops=1) Buckets: 4,096 (originally 1024) Batches: 1 (originally 1) Memory Usage: 312kB
8.	0.000	0.000	↓ 2,868.0	1	→ Parallel Hash Join (cost=713,611.17..728,105.55 rows=1 width=72) (actual rows=2,868.00 loops=1) Hash Cond: (an.person_id = n.id)
9.	0.000	0.000	↓ 4.0	1	→ Parallel Seq Scan on aka_name an (cost=0.00..13,649.36 rows=225,336 width=4) (actual rows=901,343.00 loops=1)
10.	0.000	0.000	↓ 666.0	1	→ Parallel Hash (cost=713,611.16..713,611.16 rows=1 width=84) (actual rows=666.00 loops=1) Buckets: 1,024 Batches: 1 Memory Usage: 104kB
11.	0.000	0.000	↓ 666.0	1	→ Nested Loop (cost=673,574.59..713,611.16 rows=1 width=84) (actual rows=666.00 loops=1)
12.	0.000	0.000	↓ 15,866.0		
13.	0.000	0.000	↓ 2,641,497.5		
14.	0.000	0.000	↓ 4.0		
15.	0.000	0.000	↓ 47,805.0		
16.	0.000	0.000	↓ 47,805.0		
17.	0.000	0.000	↓ 18,042.0		
18.	0.000	0.000	↓ 7,874.1		
19.	0.000	0.000	↓ 94.7		
20.	0.000	0.000	↓ 4.0		
21.	0.000	0.000	↑ 1.0		
22.	0.000	0.000	↑ 1.0		

-> Parallel Hash Join (rows=6553) (actual rows=620526)
Hash Cond: (pi.info_type_id = it3.id)
-> Parallel Seq Scan on person_info pi (rows=740474) (actual rows=2963664)
-> Parallel Hash (rows=1) (actual rows=1.00)
-> Parallel Seq Scan on info_type it3 (rows=1) (actual rows=1)
Filter: ((info)::text = 'trivia'::text)
Rows Removed by Filter: 112

Критерий 2: Селективность выборки из БД

Эффективность выборки из таблицы

```
Hash Join  (actual time=234.671 rows=289 loops=1)
  Hash Cond: (o.customer_id = c.id)
  Filter: (c.segment = 'enterprise')
  Rows Removed by Filter: 8743
  ->  Seq Scan on orders o  (actual time=98.341 rows=47831 loops=1)
        Filter: ((status = 'completed') AND (created_at >= '2024-01-01'))
        Rows Removed by Filter: 952169
  ->  Hash  (actual time=42.110 rows=100000 loops=1)
        ->  Seq Scan on customers c  (actual time=21.543 rows=100000
```

Формулы для метрики селективности ноды

Для SeqScan, IndexScan, Bitmapscan ...:

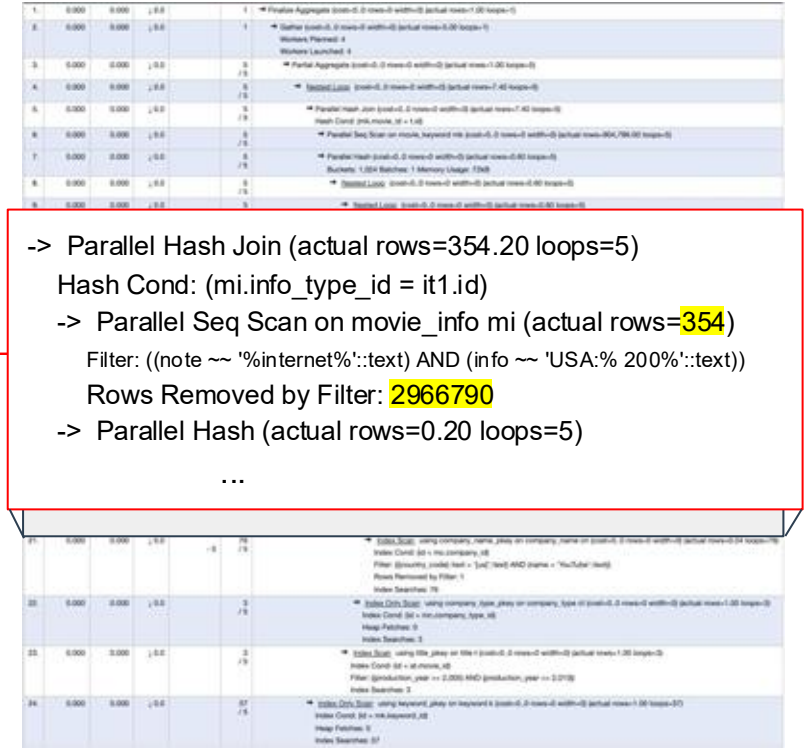
$$scan_f = \max_{scans}\left(\frac{nfiltered1}{actual_rows} \times \frac{node_exectime}{total_exectime}\right)$$

Для NestLoop, HashJoin и MergeJoin:

$$join_f = \max_{joins}\left(\frac{nfiltered1 + nfiltered2}{actual_rows} \times \frac{node_exectime}{total_exectime}\right)$$

Пример работы selectivity-критериев

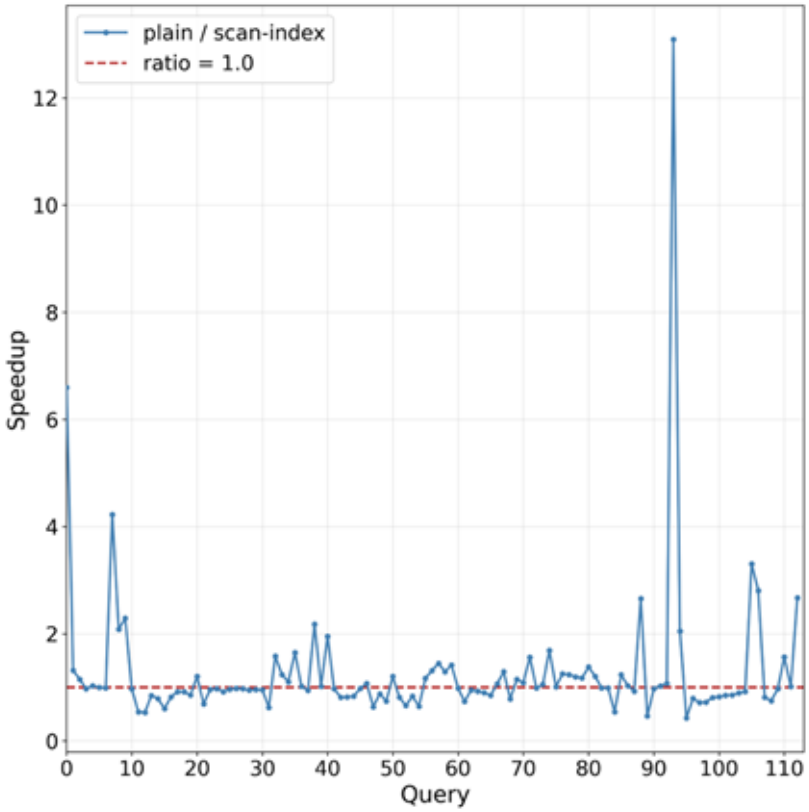
```
SELECT queryid,LEFT(query, 13) AS query,
  avg_avg AS error,lf_avg AS scan_f,
  lf_dev AS stddev
FROM track_data
ORDER BY lf_avg DESC LIMIT 10;

       queryid        |     query     | error | scan_f  | stddev
----------------------+---------------+-------+---------+--------
  7987931480126446697 | /* 15b.sql */ |  1.60 | 1459.35 |   0.00
 -4244378222084090458 | /* 23a.sql */ |  1.75 | 1424.58 |   0.00
 -6679854959453081533 | /* 15a.sql */ |  1.95 | 1406.36 |   0.00
 -1139306376856509831 | /* 23b.sql */ |  1.48 | 1403.23 |   0.00
 -4904731422305343909 | /* 15c.sql */ |  2.69 | 1342.81 |   0.00
   878606107360590202 | /* 23c.sql */ |  1.44 | 1130.56 |   0.00
   900127404096217830 | /* 3b.sql */  |  1.18 |  860.32 |   0.00
  -393358205771650517 | /* 11a.sql */ |  2.50 |  835.37 |   0.00
 -3135863217603061370 | /* 32a.sql */ |  2.32 |  580.94 | 131.71
   868681592535365242 | /* 7b.sql */  |  1.41 |  573.87 |   0.00
(10 rows)
```



-> Parallel Hash Join (actual rows=354.20 loops=5)
Hash Cond: (mi.info_type_id = it1.id)
-> Parallel Seq Scan on movie_info mi (actual rows=354)
Filter: ((note ~~ '%internet%'::text) AND (info ~~ 'USA:% 200%'::text))
Rows Removed by Filter: 2966790
-> Parallel Hash (actual rows=0.20 loops=5)
...

После создания индексов

```
       queryid         |      query
-----------------------+---------------
   9696065745275141 43 | /* 2a.sql */
 -3752195902401095348 | /* 1c.sql */
 -5865862719849412788 | /* 33a.sql */
 -7718801079997731049 | /* 1d.sql */
 -5419519412498458412 | /* 33b.sql */
(5 rows)
```

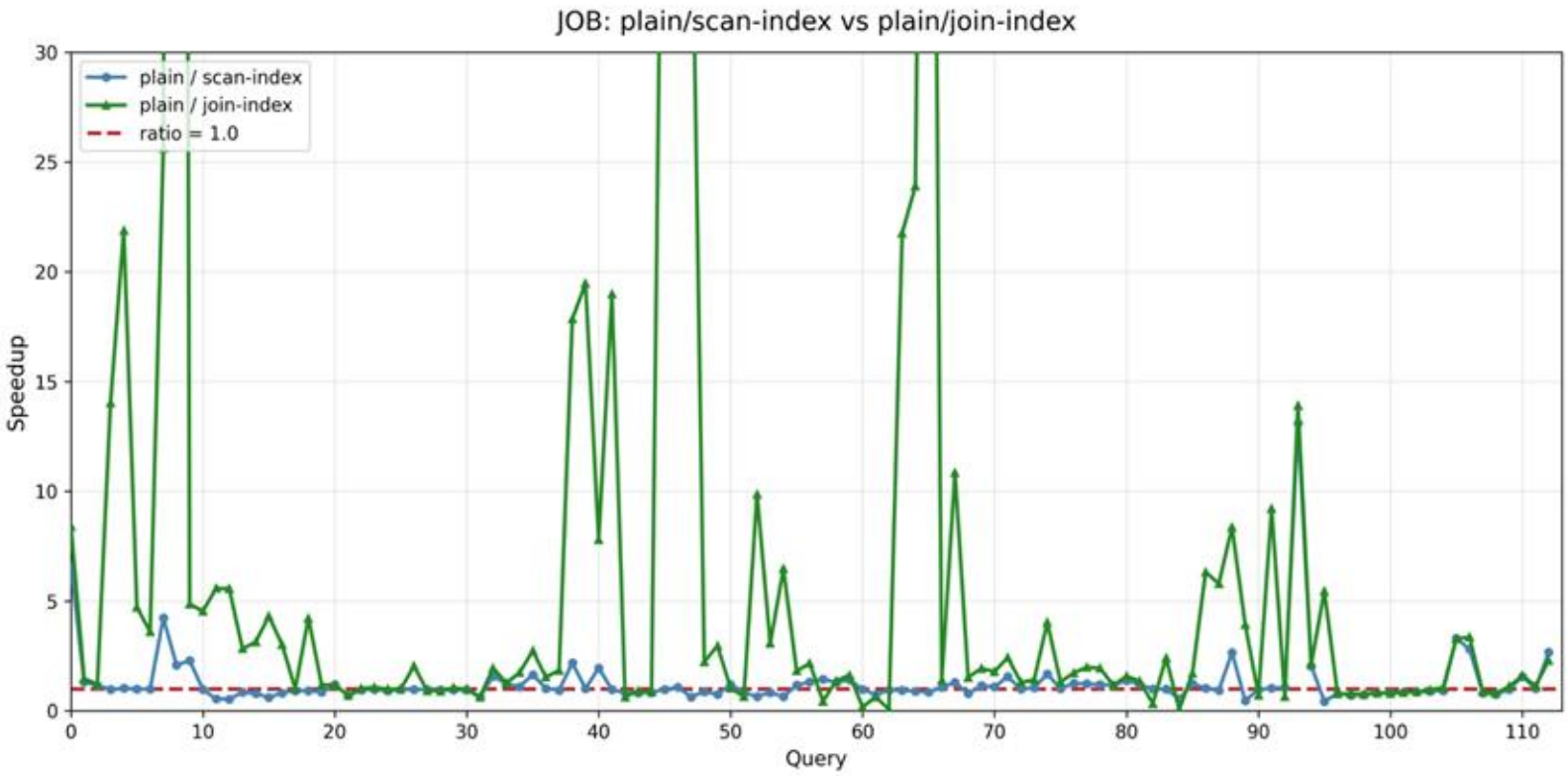


Критерий фильтрации для джойнов

```
SELECT queryid,LEFT(query, 13) AS query,
  ROUND(avg_avg::numeric,2) AS error,
  ROUND(lf_avg::numeric,2) AS scan_f,
  RO
FROM
ORDE

----
  63
 -54
 -58
  83
 -37
(5 r
```

```
Aggregate (actual rows=1.00 loops=1)
  ->  Nested Loop (actual rows=743.00 loops=1)
        Join Filter: (t.id = ml.movie_id)
        ->  Nested Loop (actual rows=743.00 loops=1)
              ->  Nested Loop (actual rows=1808.00 loops=1)
                    ->  Nested Loop (actual rows=1913.00 loops=1)
                          Join Filter: (mc.movie_id = ml.movie_id)
                          ->  Nested Loop (actual rows=251.00 loops=1)
                                ->  Merge Join (actual rows=270.00 loops=1)
                                      Merge Cond: (mk.movie_id = ml.movie_id)
                                      ->  Nested Loop (actual rows=28.00 loops=1)
                                            ->  Nested Loop (actual rows=28.00 loops=1)
                                                  Join Filter: (mk.movie_id = mi.movie_id)
                                                  Rows Removed by Join Filter: 15035457
                                                  ->  Nested Loop (actual rows=17.00 loops=1)
                                                        ->  Materialize (actual rows=884440.29 loops=17)
                                                                                                    …
```

Эффект найденных индексов



Большая часть JOIN-фильтров незаметна

```
CREATE TABLE test (x int, y int);
INSERT INTO test (x,y)
 (SELECT gs, -gs
  FROM generate_series(0,1E4) AS gs);

EXPLAIN (ANA
SELECT count
  JOIN test
```

```
Aggregate (actual rows=1 loops=1)
  ->
```

```
Aggregate (actual rows=1 loops=1)
  ->
```

```
Aggregate (actual rows=1 loops=1)
  ->  Nested Loop (actual rows=1 loops=1)
        Join Filter: (t1.x = t2.y)
        Rows Removed by Join Filter: 100020000
        ->  Seq Scan on test t1 (actual rows=10001 loops=1)
        ->  Materialize (actual rows=10001 loops=10001)
              ->  Seq Scan on test t2 (actual rows=10001 loops=1)
```

Добавим в HashJoin код для hashclause filtering*

```
Aggregate (actual rows=1 loops=1)
  ->  Hash Join (actual rows=1 loops=1)
        Hash Cond: (t1.x = t2.y)
        Rows Removed by Hash Matching: 10000
        ->  Seq Scan on test t1 (actual rows=10001 loops=1)
        ->  Hash (actual rows=10001 loops=1)
              ->  Seq Scan on test t2 (actual rows=10001 loops=1)
```

*Обсуждение и патч:

И посмотрим на активность JOIN-фильтров снова

```
SELECT queryid,LEFT(query, 13) AS query,
  ROUND(avg_avg::numeric,2) AS error,
  ROUND(lf_avg::numeric,2) AS scan_f,
  ROUND(jf_avg::numeric,2) AS join_f
FROM track_data
ORDER BY jf_avg DESC LIMIT 10;

       queryid        |     query     | error | scan_f |  join_f
----------------------+---------------+-------+--------+----------
 -3135863217603061370 | /* 32a.sql */ |  1.83 |   0.00 | 430659.68
  5025464644910963332 | /* 25b.sql */ |  1.58 |   4.77 |  61812.43
   969606574527514143 | /* 2a.sql */  |  2.87 |   0.00 |  48692.36
  8307542826537749406 | /* 5a.sql */  |  2.71 |   0.00 |  22364.01
 -8287516654825389938 | /* 12b.sql */ |  1.64 |   0.05 |  22095.91
  3345454269340541014 | /* 10b.sql */ |  3.28 |   4.76 |  21820.98
  6570380967973041608 | /* 1b.sql */  |  2.43 |   0.03 |  21817.95
 -7718801079997731049 | /* 1d.sql */  |  2.46 |   0.03 |  21348.56
   868681592535365242 | /* 7b.sql */  |  1.45 |   0.00 |  20322.10
  5164706688251065689 | /* 24b.sql */ |  1.94 |   7.30 |  11925.55
(10 rows)
```

Эффект HashJoin
hashclause filter

Критерий 3:
Спиллинг промежуточных результатов

Сброс промежуточных результатов на диск

определяется величиной work_mem как лимит для каждой ноды плана в отдельности

1. HashJoin
2. Hash Aggregate
3. Window Aggregate
4. Sort
5. Materialize
6. CTE

Отследить можно через параметр ***local_blks***
(абсолютное значение)

Посмотрим, использует ли кто спиллинг

```
SELECT queryid, LEFT(query,13) AS query,
  ROUND((temp_avg*8192/1024/1024)::numeric,0) AS spill_mem_mb
FROM track_data
ORDER BY CASE WHEN blks_avg = 0 THEN 0 ELSE temp_avg/blks_avg END DESC
LIMIT 10;
```

queryid	query	spill_mem_mb
-4733402157366146115	/* 8c.sql */	598
-3437698612165670854	/* 8d.sql */	381
4682104415514032107	/* 7c.sql */	2678
1798119524484989875	/* 10c.sql */	179
-3013169471333993680	/* 16c.sql */	91
-5865862719849412788	/* 33a.sql */	15
9035692917490887624	/* 16a.sql */	44
-8802845496176030318	/* 9b.sql */	20
3631768377448547020	/* 29c.sql */	94
-2651921679566465644	/* 9d.sql */	231

```
SET work_mem='1MB';
Execution Time: 1841 ms

SET work_mem='4MB';
Execution Time: 1405 ms

SET work_mem='64MB';
Execution Time: 950 ms
```

К чему приводит увеличение work_mem

- **Hash Join -> NestLoop + Memoize**
- IncrementalSort + MergeAppend -> Sort + Append
- MergeAppend + Sort -> Sort + Append
- **GroupAgg + Sort -> HashAgg**
- IndexScan -> Sort + SeqScan
- IndexScan -> IncrementalSort + IndexScan
- **Hash Join -> Merge Join**
- -> Material

В целом планы запросов становятся более “агрессивными”, быстрыми и чувствительными к ошибке.

Критерий 4: Стабильность планов запросов

Стабильность планов запросов

- queryid группирует запросы в классы
- pg_stat_statements позволяет обнаружить флуктуации (min,max,avg,stddev) в Execution time, Planning time.
- Имея ~23 дополнительных параметра, зависящих от плана, это увеличит ширину хранилища: 54 -> 123.
- pg_track_optimizer использует custom type RStats хранящий внутри min,max,avd,stddev

```
CREATE FUNCTION pg_stat_statements(
   IN showtext boolean,
   OUT userid oid,
   OUT dbid oid,
   OUT toplevel bool,
   OUT queryid bigint,
   OUT query text,
   OUT plans int8,
   OUT total_plan_time float8,
   OUT min_plan_time float8,
   OUT max_plan_time float8,
   OUT mean_plan_time float8,
   OUT stddev_plan_time float8,
   OUT calls int8,
   OUT total_exec_time float8,
   OUT min_exec_time float8,
   OUT max_exec_time float8,
   OUT mean_exec_time float8,
   OUT stddev_exec_time float8,
   OUT rows int8,
   OUT shared_blks_hit int8,
   OUT shared_blks_read int8,
   OUT shared_blks_dirtied int8,
   OUT shared_blks_written int8,
   OUT local_blks_hit int8,
   OUT local_blks_read int8,
   OUT local_blks_dirtied int8,
   OUT local_blks_written int8,
   OUT temp_blks_read int8,
   OUT temp_blks_written int8,
   OUT shared_blk_read_time float8,
   OUT shared_blk_write_time float8,
   OUT local_blk_read_time float8,
   OUT local_blk_write_time float8,
   OUT temp_blk_read_time float8,
   OUT temp_blk_write_time float8,
   OUT wal_records int8,
   OUT wal_fpi int8,
   OUT wal_bytes numeric,
   OUT wal_buffers_full int8,
   OUT jit_functions int8,
   OUT jit_generation_time float8,
   OUT jit_inlining_count int8,
   OUT jit_inlining_time float8,
   OUT jit_optimization_count int8,
   OUT jit_optimization_time float8,
   OUT jit_emission_count int8,
   OUT jit_emission_time float8,
   OUT jit_deform_count int8,
   OUT jit_deform_time float8,
   OUT parallel_workers_to_launch int8,
   OUT parallel_workers_launched int8,
   OUT generic_plan_calls int8,
   OUT custom_plan_calls int8,
   OUT stats_since timestamp with time zone,
   OUT minmax_stats_since timestamp with time zone
)
```

Флуктуации параметров

- ***blks_accessed, exec_time*** – базовый критерий
- ***plan_nodes, njoins*** - разные запросы в одном queryid или прунинг
- ***evaluated_nodes*** - запрос чувствителен к тюнингу

Хороший кандидат на generic план:

- Неизменный *njoins*, *plan_nodes*. Значение *blks_accessed* колеблется незначительно

```
CREATE FUNCTION pg_track_optimizer(
   OUT dboid           Oid,
   OUT queryid         bigint,
   OUT query           text,
   OUT avg_error       rstats,
   OUT rms_error       rstats,
   OUT twa_error       rstats,
   OUT wca_error       rstats,
   OUT blks_accessed   rstats,
   OUT temp_blks       rstats,
   OUT exec_time       rstats,
   OUT f_join_filter   rstats,
   OUT f_scan_filter   rstats,
   OUT f_worst_splan   rstats,
   OUT njoins          rstats,
   OUT evaluated_nodes integer,
   OUT plan_nodes      integer,
   OUT nexecs          bigint
)
```

Критерий 5:
Влияние SubPlan'ов

Критерий поиска плохих сабпланов

$$sf = \max_{nodes}\left(\frac{nloops}{\log(nloops+1)} \times \frac{subplan_time}{total_exectime}\right)$$

Что такое RStats?

пользовательский тип PostgreSQL для накопления статистики без потери числовой точности

СТРУКТУРА (C)

```c
typedef struct RStats {
  int64  count;  /* N наблюдений */
  double mean;   /* текущее среднее */
  double m2;     /* сумма кв. откл. */
  double min, max;
} RStats;  /* 40 байт, fixed-size */
```

Алгоритм Уэлфорда — инкрементальное обновление среднего и дисперсии без накопления всех значений. Высокая числовая стабильность при больших N.

SQL ИНТЕРФЕЙС

```sql
-- оператор + накапливает значения
SELECT 10::rstats + 20 + 30;
-- (count:3, mean:20, min:10, max:30, stddev:8.16)

-- доступ к полям через '->'
SELECT avg_error->'mean',
       avg_error->'stddev',
       avg_error->'count'
FROM pg_track_optimizer();

-- пустое состояние = нода не исполнялась
SELECT rstats_is_empty(avg_error);
-- true  ⟹  count=0, данных нет
-- false ⟹  count>0, статистика накоплена
```

Тип RStats

унификация сбора статистики и решение проблемы «неиспользуемого» запроса

БЫЛО — 55+ столбцов

```
-- для каждой из 11 метрик:
avg_error_count  bigint,
avg_error_mean   float8,
avg_error_stddev float8,
avg_error_min    float8,
avg_error_max    float8,

-- ... ещё 47 строк

-- NULL если статистика не определена для плана
запроса
```

count = 0 — параметр не определён

→

СТАЛО — 11 столбцов

```
-- одна метрика = один столбец:
avg_error     rstats,
rms_error     rstats,
twa_error     rstats,
blks_accessed rstats,
exec_time     rstats,  -- ...6 more


-- rstats = { count, mean, stddev, min, max }
-- count=0  →  нода не исполнялась
-- нет NULL, нет неопределённых значений
```

Один тип — единый код агрегации, единый UI,
55 столбцов → 11 (–80%)

Что мониторить

1. ***Average error*** - может сигнализировать о снижении эффективности системы и необходимости обновления статистики
2. ***Max error*** - может означать, что в системе появляются ad-hoc запросы, которые в какой-то момент могут зависнуть в долгом выполнении
3. **Custom plans** которые могли бы стать generic: *pg_stat_statements JOIN pg_track_optimizer ON queryid WHERE blocks_accessed WITHIN 10%*
4. **Generic plans** которым лучше стать custom

User Interface

- RStats type, rstats_agg()
- FUNCTION pg_track_optimizer()
- VIEW pg_track_optimizer
- FUNCTION pg_track_optimizer_flush()
- pg_track_optimizer_reset()
- GUCs:
  - log_min_error
  - hash_mem
  - auto_flush

Микробонус: состояние расширения

```
SELECT * FROM pg_track_optimizer_status;
   mode    | entries_left | is_synced
-----------+--------------+-----------
 disabled |         8642 | t
```

Спасибо за внимание!

Отзывы и сообщения об ошибках are welcome!

lepihov@gmail.com

github.com/danolivo

pgedge.com

pg_track_optimizer

Отслеживание ошибок оценки кардинальности в планах запросов PostgreSQL

danolivo/pg_track_optimizer

PostgreSQL 17+ · ~1–2% накладных расходов

PG BootCamp Russia 2026 Moscow

PGBootCamp.ru